

UNIVERSITY COLLEGE LONDON
DEPARTMENT OF PHYSICS AND ASTRONOMY

Tensor Networks for Classical and Quantum Machine Learning

Author:
Aleksej Rodano

Supervisor:
Prof. Andrew G. Green

*A dissertation submitted in partial fulfillment of the
requirements for the degree of*
MSci Theoretical Physics

April 13, 2026

Word Count: 6300
Excluding References and Appendix (Overleaf word count)

Abstract

The deterministic tensor network classifier of Wright et al. [1] achieves modest classification accuracy on MNIST. This falls below that of standard machine learning models such as convolutional neural networks, which achieve above 99% accuracy on the same dataset. This report investigates whether the classifier can offer transparency into its decision-making process, and whether understanding its internal structure can lead to a more efficient construction.

This classifier is built deterministically using Quantics data encoding. This allows the classifier to organise hierarchically, from global features to pixel-level details. Feature extraction reveals that fine-scale 4×4 geometric primitives (motifs) show high cross-class overlap, exceeding 0.99 for the majority of classes. Replacing the fine-scale tensors of all ten classes with a single shared motif drops accuracy from 80.9% to 80.4%, confirming a redundancy in the stored data.

The study also sets up an algebraic framework for manipulating Quantics encoded motifs. It is shown that rotating and scaling a motif are cheap operations that do not increase the complexity of the representation. Shifting a motif by a single pixel is far more complex. For this manipulation an MPO with bond dimension two is required, regardless of grid size. Furthermore, when multiple instances of the same motif appear at different positions, the identical parts of their representations can be stored only once. This achieves lossless compression.

Together, these two results point toward a more efficient and interpretable classifier, where a single shared motif and a set of geometric placement rules are enough to reconstruct the full classifier.

Contents

1	Introduction	3
1.1	Motivation – Interpretability of Machine Learning Based on Tensor Networks . . .	3
1.2	Objective - Analytic Feature Extraction and Geometric Manipulation	3
2	Tensor Network Foundation	3
2.1	Notation	3
2.2	Matrix Product State (MPS) Representation	4
2.3	Canonical Forms of MPS	5
3	Quantum Data Encoding	5
3.1	Basis Encoding	5
3.2	Amplitude Encoding	5
3.3	Quantics Encoding: Hierarchical Feature Mapping	6
4	Deterministic construction of Quantics MPO classifier	7
5	Internal Structure and Compressibility of the Quantics MPO Classifier	8
5.1	Extracting Class Masks	8
5.2	Motif Decomposition	9
5.3	Scale-Dependent Compressibility of the Classifier	10
6	Geometric Operators on Quantics-Encoded States	11
6.1	Motif and Their Basis Representation	12
6.2	Constructing MPS from Geometric Primitives	12
6.3	Motif Manipulation: Rotation, Scaling and Quadrant Size Translation	13
6.4	Single-Pixel Translation	14
6.4.1	Factorisation and Why Single-Pixel Translation Breaks It	14

6.4.2	Constructing the Single-Pixel Translation MPO	15
6.5	Suffix Compressibility of Images Constructed from Motifs	16
7	Discussion	17
7.1	Limitations	18
8	Conclusion	18
8.1	Improvements and Future Work	18
8.2	Acknowledgement	19
Appendix A	From SVD to Matrix Product State	22
Appendix A.1	Singular Value Decomposition	22
Appendix A.2	Constructing MPS	22
Appendix B	MPS Summation and Maximum Bond Dimension	22
Appendix B.1	Block-Diagonal Summation of MPS	22
Appendix B.2	Maximum Theoretical bond dimension in the Quantics MPO Classifier	23
Appendix C	8×8 Masks Extracted from the Quantics MPO Classifier	23
Appendix D	MPO Construction for Single-Pixel Translation via Carry Arithmetic	25

1. Introduction

1.1. Motivation – Interpretability of Machine Learning Based on Tensor Networks

Standard machine learning models can classify data exceptionally well but often lack insight into why the classifier made a particular decision [2, 3]. These models are optimised over millions of parameters and as such they are treated as black box models. This lack of transparency is a problem. Without this understanding, it is hard to trust or improve the model [4].

Tensor networks were originally developed to study many-body quantum systems [5, 6, 7]. Their use in machine learning is motivated by the fact that they decompose high-dimensional data into structured networks of smaller interconnected elements. The classifier construction in Wright et al. [1] is built on this principle. However, it is not performing as well as hoped and achieves modest accuracy (below 85%) on MNIST dataset classification. A question is whether this classifier could be built more efficiently by gaining a deeper understanding of its internal components and how to manipulate them.

Images are naturally structured at many different scales [8, 9]. In an image, pixels group together to form edges, edges combine to create shapes, and shapes are combined into objects. This kind of multi-scale hierarchical structure is a fundamental property also present in linguistic data, where individual words combine into phrases, and phrases combine into sentences. The Wright et al. classifier studied in this project encodes images in a way that replicates this hierarchy, storing information from coarse to fine features. Because of this encoding scheme, there is reason to believe that the internal components might correspond to features a human could interpret. Understanding them and knowing the rules for manipulating them could be used to improve the classifier’s performance.

Studying tensor networks for machine learning has a value beyond the classification task. Tensor networks have been used to compress large-scale neural networks. For example, applying tensor network compression to LLaMA-2 7B reduces its memory footprint by 93% and its parameter count by 70%, with an accuracy drop of only 2–3% [10]. A further motivation for studying them is their efficient implementation on quantum hardware [11, 12]. The structures they rely on correspond directly to sequences of quantum gates. This makes the classifier studied in this work a natural candidate for future quantum hardware implementation.

1.2. Objective - Analytic Feature Extraction and Geometric Manipulation

This work addresses two questions. First, does the classifier store recognisable geometric features? Are these features shared across classes? Second, can we translate, rotate, or rescale these geometric primitives? Can these be done without increasing the complexity of the representation?

2. Tensor Network Foundation

Tensor networks provide a way to break down high-dimensional data into smaller, connected parts. This section defines the mathematical language used to describe these structures.

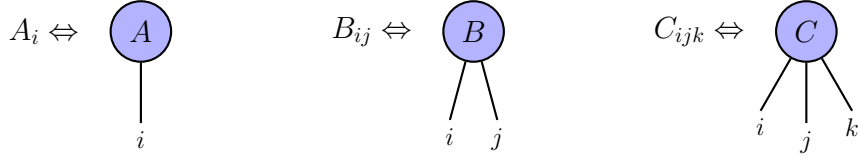
2.1. Notation

Tensors are a general way to represent data. They include scalars, vectors, and matrices. A scalar has no indices, a vector has one, a matrix has two, and a rank- n tensor has n indices, with its components written as $T_{i_1 i_2 \dots i_n}$. Each index runs over a set of values determined by its dimension. Taking the tensor product of two tensors raises the total rank, i.e. $A_i \otimes B_j = C_{ij}$.

Tensor networks are a mathematical scheme originally developed in quantum physics to compactly represent the states of many-body quantum systems. This involves decomposing a high-dimensional state into a network of smaller, interconnected tensors. These connections are represented as the index contraction between them, and they represent the entanglement between sub-systems.

Working with many indices becomes very hard to follow using standard notation. Penrose notation solves this problem [5, 6, 7, 13]; it represents equations with diagrams (Fig. 1). Each

a)



b)

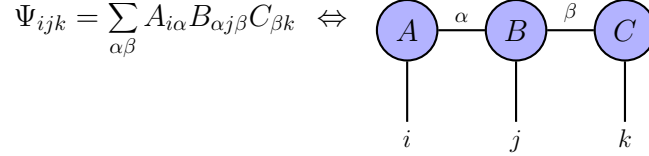


Figure 1: **Penrose notation.**

In this notation, tensors are represented as shapes and each index is shown as a leg. **a)** From left to right: a vector, a matrix, and a rank-3 tensor. **b)** A Matrix Product State (MPS) is a one-dimensional chain of tensors. This is used to represent quantum systems. The vertical legs represent the physical dimension d , which corresponds to the observed state of each site. The horizontal lines connecting the tensors represent the bond dimension χ . These bonds indicate contracted indices that carry the entanglement between neighboring sites.

tensor is drawn as a shape with legs coming out of it. Legs can represent two things: a free leg represents an uncontracted index, and a leg connecting two tensors is a summation over a shared index – also known as a bond. The number of legs attached to a tensor equals the rank of the tensor.

2.2. Matrix Product State (MPS) Representation

A quantum system of N sites lives in a Hilbert space that grows exponentially with N . The joint state of the system is $|\psi\rangle = |\psi\rangle^{[1]} \otimes |\psi\rangle^{[2]} \otimes \dots \otimes |\psi\rangle^{[N]}$. If each local state has dimension d , then the total dimension of this space is d^N . A wavefunction can be equivalently represented as a high-rank tensor $\psi_{\sigma_1, \sigma_2, \dots, \sigma_N}$, where the indices σ_k run over d states. For arbitrarily large systems it will be practically impossible to compute and store these systems. Hence it is required to use a technique that decreases the dimensionality of the system without significant loss of information about it.

The idea is to decompose this high-rank tensor into a chain of lower-rank tensors using repeated Singular Value Decomposition (SVD) (full details in [Appendix A.1](#)) [5, 14]. SVD splits any matrix M as $M = USV^\dagger$ where S is diagonal and contains the singular values in descending order [15]. U and $V^\dagger$ are matrices with orthonormal columns and rows, respectively. Each singular value measures how much information a particular component carries. Removing the smallest singular values gives a compressed approximation. Applying this decomposition repeatedly, site by site, breaks the full state tensor into a 1D chain tensor network, also known as Matrix Product State (MPS) Fig. 1 (b) (full derivation in [Appendix A.2](#)) [16]. The tensors in the MPS have 3 indices each: one physical index σ_k encoding the local state at site k , and two bond indices connecting to the neighbouring tensors. The tensors at the boundaries of the chain are rank-2, since they have only one neighbour.

The approximation of the resulting state is controlled by the size of the bond dimension χ , which measures how much correlation is allowed between neighbouring sites. Truncating to a maximum bond dimension $\chi_{\max}$ discards the weakest correlations. This reduces storage from d^N to $\mathcal{O}(dN\chi^2)$, which is exponentially cheaper [17, 18].

Finally, operators acting on an MPS are represented as Matrix Product Operators (MPOs), where each site tensor carries an input and an output physical index, with the site tensor having indices $(\chi_{k-1}, \chi_k, \sigma, \sigma')$.

2.3. Canonical Forms of MPS

An MPS has a gauge freedom, as the site tensors can be transformed in many ways without changing the physical state they represent. This implies infinitely many representations exist for the same state. Choosing a canonical form fixes this gauge and gives the MPS well-defined properties [17].

An MPS is called right-isometric if contracting it with its conjugate over the physical and right bond indices gives the identity [19]. Similarly, a tensor is called left-isometric if contracting it with its conjugate over the physical and left bond indices gives the identity.

The mixed-canonical MPS is constructed so that it contains a single non-isometric tensor, called the orthogonality centre. The tensors to the right of the orthogonality centre are right-isometric and tensors to the left of it are in left isometry. This construction concentrates all the information of the state into the orthogonality centre. A key consequence of this form is that the bond indices entering the orthogonality centre are ordered by decreasing singular value. This ordering makes it straightforward to identify which correlations are the most important.

3. Quantum Data Encoding

To construct a tensor network classifier, the classical data must first be converted into a quantum state. This process is called quantum data encoding. There are several ways to do this, and the choice of encoding has an impact on how well the classifier can learn and how interpretable the resulting representation is. This section describes three encoding strategies: basis encoding, amplitude encoding, and Quantics encoding.

3.1. Basis Encoding

The simplest approach is basis encoding [20, 21]. To encode continuous pixel intensities, the value must first be discretised into a binary string. Each bit of this string is then mapped directly to a corresponding qubit in the computational basis, where a classical bit 0 becomes $|0\rangle$ and 1 becomes $|1\rangle$.

Because each pixel is represented by a multi-bit string, an image of N pixels requires $N \times d$ qubits, where d is the bit length. While this encoding is straightforward to implement, it is very inefficient; the number of qubits grows linearly with the number of pixels. An additional reason not to use this encoding is that the structure of the encoded state carries no useful information about the relationships between pixels.

3.2. Amplitude Encoding

A more efficient approach is amplitude encoding [20, 21, 22]. Here, the pixel intensities of an image are stored as the amplitudes of a quantum state. Each pixel is labelled by a binary bit string $|b\rangle$ basis, and the encoded state is:

$$|\Psi\rangle = \sum_b a_b |b\rangle, \quad (1)$$

where a_b is the intensity of the pixel corresponding to bit string b . In this encoding images are stored as a superposition of all basis states, and only $\log_2(N_{pixels})$ qubits are required to represent an image of N_{pixels} pixels [21]. This is an exponential improvement compared to basis encoding.

Standard amplitude encoding has a limitation. The procedure flattens an image from 2D into a 1D vector by scanning row by row. The resulting MPS built from the flat scan does not organise features from coarse to fine spatial scales. This property is key to the investigation of this work; hence a different scanning method is required.

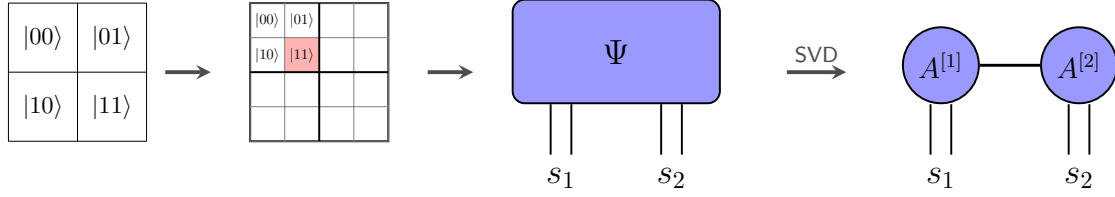


Figure 2: **Quantics encoding of an image into an MPS.**

An image is divided into four quadrants. Each quadrant is labelled by a two-bit string $s_1 \in \{00, 01, 10, 11\}$, which will represent the first site in the MPS. Every quadrant is then subdivided again, with the new sections labelled by a second string s_2 . Each pixel now has a unique address (s_1, s_2) , and its intensity is assigned as the amplitude of the corresponding basis state $|s_1\rangle \otimes |s_2\rangle$. The full image is consequently encoded as the superposition $|\Psi\rangle = \sum_{s_1, s_2} I(s_1, s_2) |s_1\rangle \otimes |s_2\rangle$, whose coefficient array is a rank-4 tensor of shape $2 \times 2 \times 2 \times 2$. An SVD converts this tensor into a 2-site MPS with physical dimension 4. For example, the red pixel, in position $(00, 11)$, is encoded as the state $a|00\rangle \otimes |11\rangle$, where a is its intensity.

3.3. Quantics Encoding: Hierarchical Feature Mapping

Quantics encoding is a special type of amplitude encoding that solves this problem. It reorders the pixels so that the MPS chain naturally organises image features from coarse to fine, with the leftmost site capturing global structure and the rightmost site encoding fine pixel-level detail.

The encoding works by recursively subdividing the image into quadrants (Fig. 2) [23]. At each level k , every region is split into four sub-regions, each labelled by a two-bit string $s_k \in \{00, 01, 10, 11\}$, where the two bits encode the row and column choice at that level. To encode an image that is $2^N \times 2^N$, the subdivision is repeated N times such that the final division contains exactly one pixel per quadrant. This way each pixel is uniquely addressed by the string $(s_1, \dots, s_N)$.

Each pixel intensity $I(s_1, \dots, s_N)$ is then assigned as the amplitude of the corresponding basis state, giving the encoded image state:

$$|\Psi\rangle = \sum_{s_1, \dots, s_N} I(s_1, \dots, s_N) |s_1\rangle \otimes \dots \otimes |s_N\rangle \quad (2)$$

which is a tensor with N indices, each of dimension 4. Applying successive SVDs to this converts it into an MPS chain, where each level of subdivision corresponds to a site with physical dimension 4, two for the row bit and two for the column bit [24].

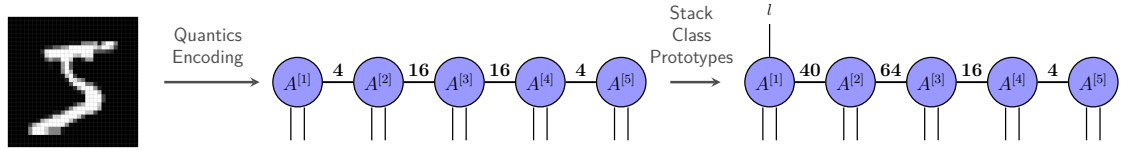


Figure 3: **Pipeline for Constructing a Quantics MPO Classifier.**

Methodology adapted from [1]. The MNIST training images, of size 28×28 pixels, are scaled to 32×32 and then Quantics encoded. This converts each image into an MPS with 5 sites. This approach allows the leftmost site to capture coarse features while the rightmost distinguishes pixel-level detail. The bond dimension between sites is illustrated above each bond. MPS within a class are then summed to create a prototype MPS for each class. Finally, the 10 class prototypes are stacked to form a single matrix product operator. When contracted with a test MPS, this MPO returns a vector of 10 class scores.

4. Deterministic construction of Quantics MPO classifier

The classifier used in this work is a matrix product operator, built entirely from linear algebra operations. The methodology is taken from Wright et al. [1]. Unlike most machine learning models, this classifier requires no gradient descent [25] and no random parameter initialisation. The construction is based on tensor network compression and quantics data encoding, hence it is fully deterministic.

This section provides a high-level description of the construction of the Quantics MPO classifier. This is applied to the MNIST dataset, which is a collection of handwritten digits going from 0 to 9. The construction is carried out mainly in two steps (see Fig. 3).

Building class prototypes: In the first step, the training images are converted to MPS via the Quantics encoding. Then, each class is summarised by a single representative MPS called a *prototype*. The prototype is built by stacking the tensors in each MPS in a block-diagonal scheme (scheme described in Appendix B.1). The resulting MPS has a large bond dimension, therefore is compressed using SVD to retain only the dominant features. The prototype captures the average spatial fingerprint of its class, and one prototype is produced per class.

Assembling the classifier: In the second step, the ten class prototypes are stacked into a single MPO by adding a label index l on the first site. The stacking uses the same block-diagonal scheme as the prototype summation step. To achieve the unitary form required for quantum circuit implementation, a polar decomposition is applied. A polar decomposition is a singular value decomposition where singular values are replaced with the identity matrix. This yields the best unitary approximation and forces the resulting states into an orthonormal set. The theoretical maximum bond profile of the resulting MPO for 32×32 MNIST images is $(40, 64, 16, 4)$ (full explanation in Appendix B.2).

To classify a test image, the image is first quantics encoded as an MPS, and then the overlap with the MPO is computed. This returns a vector of ten class scores, one overlap value per class, the predicted class is the one with the highest score.

Redundancy in the representation: An important finding noted in Wright et al. [1] is that peak accuracy is reached at a bond dimension of approximately 20, which is well below the theoretical maximum. This means most of the information retained at the full bond dimension is redundant. The following sections investigate whether the components in the classifier encode recognisable spatial patterns at each scale.

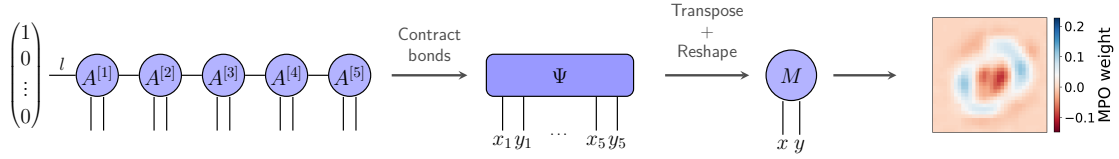


Figure 4: **Extraction of a class mask from the Quantics MPO classifier.**

Contracting the label index l with a one-hot vector selects a single class in the classifier. This collapses the MPO into a single-class MPS prototype. Contracting all the virtual bonds merges the chain into a single state Ψ carrying the ten physical indices $(x_1, y_1, \dots, x_5, y_5)$. Transpose this into $(x_1, \dots, x_5, y_1, \dots, y_5)$ and then reshape it so that x indices and y indices are grouped together. The result is a $2^5 \times 2^5$ matrix which is then plotted to produce a 32×32 image — the class mask. This represents the average spatial pattern learned by the classifier for that class.

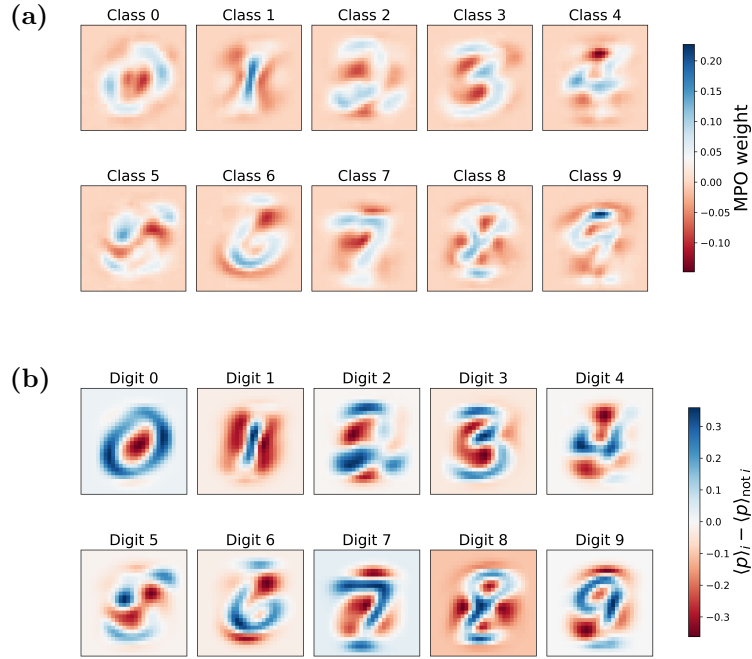


Figure 5: (a) **Class masks reconstructed from the Quantics MPO classifier.**

Each mask shows the regions the classifier weights the most heavily when recognising that class.

(b) **Pixel-wise mean difference masks.**

Each mask is computed from the raw pixel values of the training MNIST images, with no machine learning involved. For each class i , the average pixel intensity across all images of that class is calculated, and then the average pixel intensity across all images not belonging to that class is subtracted from it. This visualisation method follows from [26]. The strong similarities between the learned features in (a) and the statistical baselines in (b) suggest that the classifier relies on features consistent with basic class-wise pixel distributions.

5. Internal Structure and Compressibility of the Quantics MPO Classifier

This section investigates what features the classifier has learned. First, the class masks are reconstructed. A mask for a specific class illustrates what the ideal image looks like for that specific class. Then, the small-scale features are extracted and the similarity of these features across classes is investigated.

5.1. Extracting Class Masks

Per-class MPS prototype can be retrieved from the classifier by contracting the label index with a one-hot vector (Fig. 4). Contracting all virtual bonds merges the 1D chain into a single tensor with 10 indices. Transposing and reshaping the tensor into a 32×32 image will represent the mask for the class selected by the one-hot vector. The 10 class masks are plotted in Fig. 5 (a).

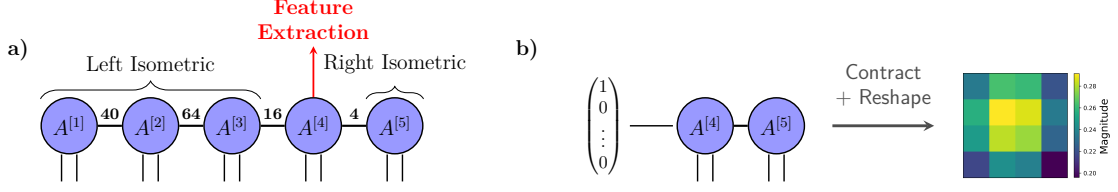


Figure 6: **Extraction of the dominant 4×4 motifs from the MPS prototypes.**

(a) The per-class prototype MPS is converted into mixed-canonical form with the orthogonality centre at site 4. Sites 1–3 are left-isometric, so the virtual bond entering site 4 is ordered by decreasing singular value. (b) Contracting the one-hot vector $v_1 = (10 \dots 0)$ with the left virtual bond of $A^{[4]}$ selects the dominant bond. After contracting the shared bond, transposing and reshaping the tensor, the result yields the dominant 4×4 motif.

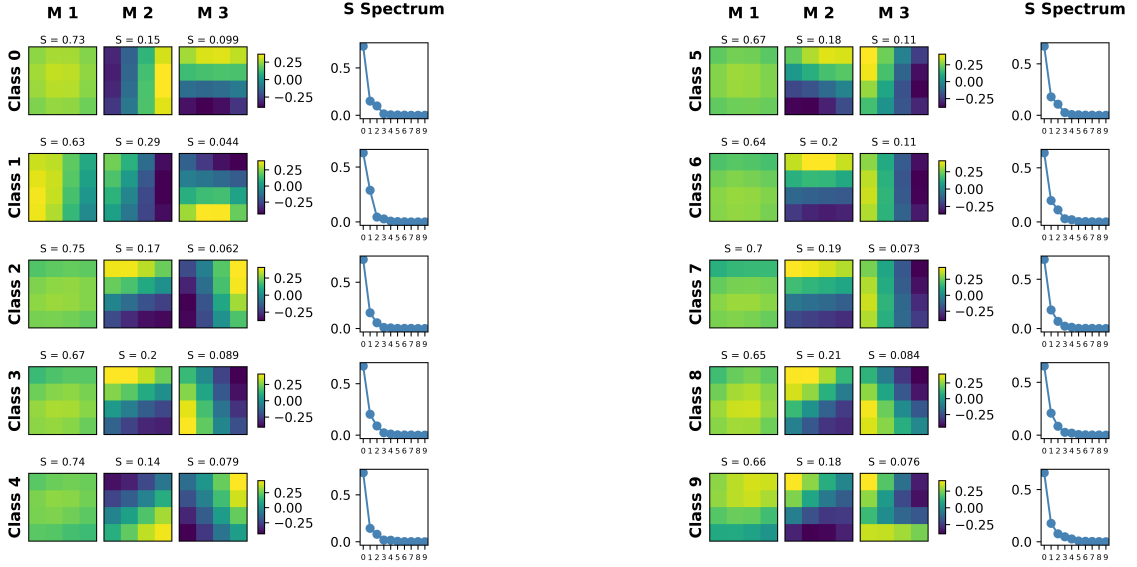


Figure 7: **Top three 4×4 motifs per-class.**

The motifs are obtained by transforming each class-specific prototypical MPS into mixed-canonical form, centred at site 4. This way, the left bond to site 4 carries the singular value spectrum $\{S_n\}$ (values are shown in the rightmost column), where each index n corresponds to a specific 4×4 spatial pattern. The first three motifs (corresponding to the largest singular values) are shown for each class. A rapid decay of the singular values means that the class is almost entirely described by its first few motifs, while a slower decay indicates that multiple patterns are needed.

The class masks closely match a statistical baseline computed directly from the training data. The statistical baseline is created by averaging the pixel intensity of all images in class i , minus the average pixel intensity of images not in class i , the result is illustrated in Fig. 5 (b). The cosine similarity, which is the overlap between the flattened images divided by their magnitudes, is above 0.70, and for some digits above 0.90. This confirms that the classifier has learned similar features.

5.2. Motif Decomposition

Converting the per-class prototype MPS from right-canonical to mixed-canonical form reveals which spatial patterns dominate at each scale. Setting the orthogonality centre at site k orders the virtual bond entering that site by decreasing singular value (Fig. 6 (a)). The features are extracted by contracting the left bond of the orthogonality centre with one-hot vectors (Fig. 6 (b)).

Extracting 4×4 motifs: The first investigated scale is the one that creates 4×4 motifs. This is constructed by setting site 4 of each prototype MPS to be the orthogonality centre. The left bond of site 4 has dimensionality 16, meaning that up to 16 motifs exist at this scale. Because the singular values for this site decrease very rapidly, only the first three 4×4 motifs are selected and plotted in Fig. 7.

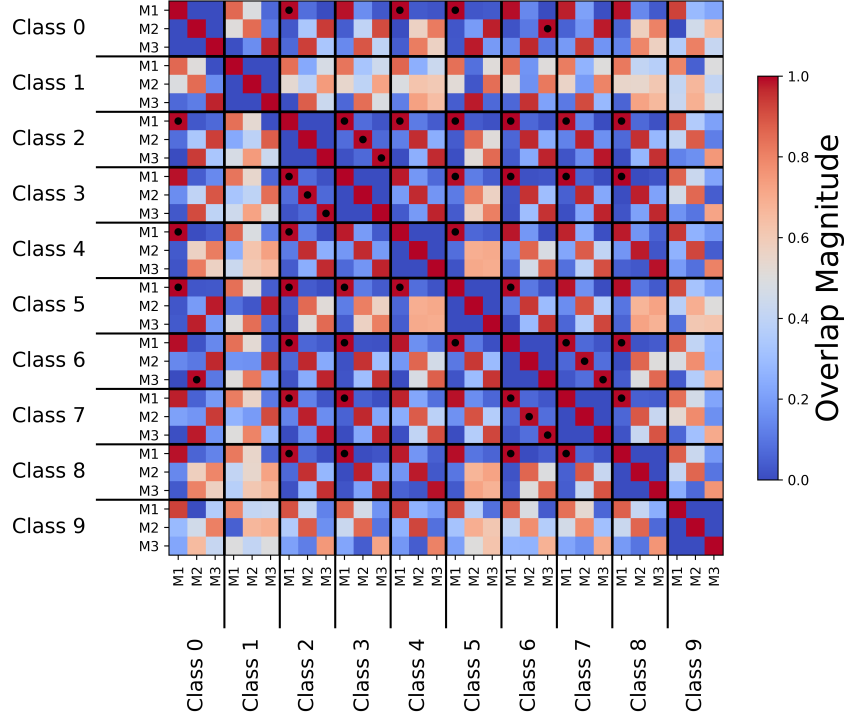


Figure 8: 4×4 **motif overlap**.

Heatmap of the overlap scores between motifs reconstructed from the last 2 sites of the MPS prototype. Only the 3 most contributing motifs per class have been selected, denoted as M1, M2, and M3. The overlap is computed by vectorising the 4×4 images and then computing the dot product over their magnitudes. Motifs with overlap higher than 0.99 are denoted by a black dot (excluding the diagonal, since it is trivially $\text{diag}(1, \dots, 1)$).

Motif cross-class similarity: The singular values decay rapidly for each class, meaning the first few motifs capture most of the information. The majority of these motifs have a high overlap across classes (Fig. 8), meaning that most classes share near-identical fine-scale features. There is a general trend of motifs from classes 1 and 9 having small overlap with all other motifs. Note that the third motifs from class 2 and class 4 give a high overlap, however looking at Fig. 7 the motifs do not look alike at all. This is caused by the fact that the SVD decomposition has an arbitrary sign flip. Hence the magnitudes of these two motifs have opposite signs but the spatial feature is the same.

Extracting 8×8 masks: The second investigated scale is the 8×8 features. These are extracted from the coarser site, where site 3 of the MPS is the orthogonality centre. In contrast to the 4×4 motifs, there is no overlap across classes (features and overlap are shown in Appendix C). As expected, this suggests that coarse sites encode the global shape of each digit, which is class-specific, while fine sites encode pixel-level features that are universal.

5.3. Scale-Dependent Compressibility of the Classifier

This section analyses further which sites carry class-discriminating information and which are redundant. The baseline accuracy is set to be the accuracy of the MPO that has a maximum bond dimension of 20, hence a bond profile of $(20, 20, 16, 4)$. For all benchmarks, 3000 test images have been used. The accuracy of this model is 81.3%.

Grid search for the most compact representation: Each bond dimension of each per-class MPS is truncated independently. The most compact bond profile is found by a brute-force grid search over all bond profiles $(\chi_1, \chi_2, \chi_3, \chi_4)$ with $\chi \in \{1, \dots, 5\}$. The compactness of each configuration is measured by its total parameter count, computed as $\sum_{k=1}^N d \cdot \chi_{k-1} \cdot \chi_k$, where $d = 4$ is the physical dimension and $\chi_0 = \chi_N = 1$. The bond profile configurations retained are the ones that have the

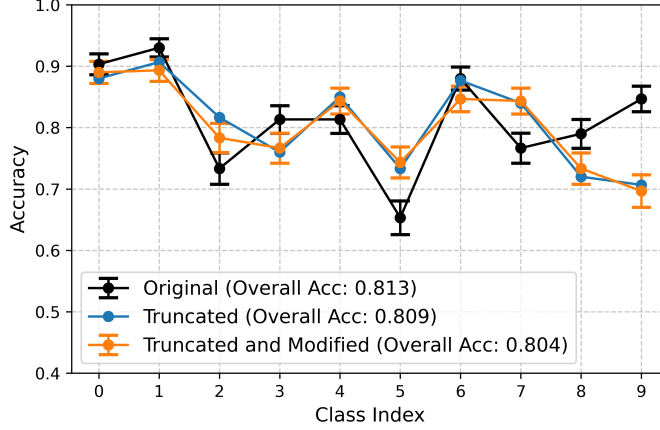


Figure 9: **Per-class accuracy comparison after truncation and motif replacement.**

Original: The classifier is a single MPO with bond profile (20,20,16,4). **Truncated:** Each per-class MPS is extracted from the MPO. Each MPS is then compressed independently and truncated to the bond profile (4,4,1,1). Inference is performed by evaluating each MPS overlap separately and then taking the argmax. **Truncated and modified:** This ensemble has the same compression as the previous model, with an additional replacement of the last two sites of every class MPS with those of class 0. This forces all classes to share an identical 4×4 motif. Overall accuracy drops by less than 1%, confirming that fine-scale features carry no class-discriminating information. Standard error bars (computed from binomial statistics over 3000 test images) are shown for the original and modified model. The truncated accuracies error bars have similar magnitudes to those of the other two models, so they are omitted for clarity.

testing accuracy within 0.5% of the baseline accuracy. The most compact bond profile is given by (4,3,2,1) with an accuracy of 81.4%.

Asymmetric compression: The classifier does not compress uniformly across all sites. The rightmost bond compresses to $\chi = 1$, which implies that at the finest scale one motif suffices to represent the data. In the grid search carried out, the leftmost two bonds never drop below $\chi = 3$. This is because truncating them would remove class-discriminating features and, consequently, drastically decrease the accuracy of the classifier.

Forcing a shared motif across all classes: Motifs from Section 5.2 proved to have a high overlap across all classes. To investigate the accuracy of forcing these motifs to be identical across all classes, the bond profile (4,4,1,1) per MPS was used, which has an accuracy of 80.9%. This sequence of truncations has been used for two reasons. Firstly, it is the second most compact configuration in the grid search. Secondly, it already has one 4×4 motif per class. After replacing the site 4 and site 5 tensors of all other classes with those of class 0, the accuracy dropped by only 0.5%. The accuracy per-class is illustrated in Fig. 9. The standard error is estimated from binomial statistics, $SE = \sqrt{p(1-p)/n}$, where $n = 3000$ test images. The small drop in accuracy implies that the same 4×4 motif can be used to represent all classes.

Truncation removes noise: Truncating the per-class MPS improves the accuracy for classes 2, 5, and 7. This indicates that the components with the smallest singular values carry noise and no useful signal for some classes.

6. Geometric Operators on Quantics-Encoded States

Training images are hypothesised to be composed of repeated geometric primitives [8, 9]. The idea is that they appear at different positions, orientations, and scales across the image. This section derives the framework for manipulating geometric primitives within the quantics MPS representation. Four different manipulations have been considered and are summarised in Figures 12, 13 and 14.

Two main results have been established. The first is that rotation, scaling, and quadrant translation are all "free" operations. They can be performed without increasing the bond dimension.

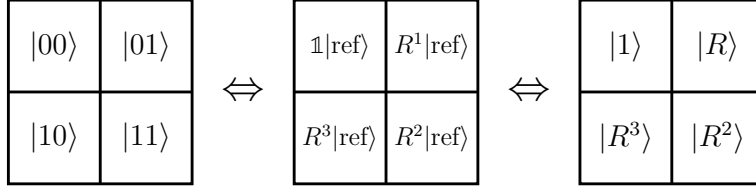


Figure 10: **Basis labelling of the Quantics MPS.**

In the Quantics encoding at every site there are four possible pixel positions within a 2×2 cell. **(Left)** Each position is labelled by a two-bit row-column string. **(Centre)** The same four positions can be generated by repeatedly applying a 90° clockwise rotation operator R to the top-left reference pixel $|\text{ref}\rangle$. **(Right)** The shorthand $|R^k\rangle$ denotes the position reached after k rotations.

The second one is that single-pixel translation forces inter-site entanglement (hence bond dimension increases) and requires a dedicated MPO.

Before any of these operations can be defined, a basis for describing motifs within a Quantics MPS is needed.

6.1. Motif and Their Basis Representation

Previously it was established that Quantics MPS labelled its physical indices via bit strings $(|00\rangle, |01\rangle, |10\rangle, |11\rangle)$. This labelling can be equally generated by repeatedly applying a rotation operator R to a single reference state $|\text{ref}\rangle$ (Fig. 10). This makes rotational symmetry a natural feature of the basis.

Let us define the reference state $|\text{ref}\rangle$ as the pixel occupying the top left quadrant in the 2×2 cell. The operation matrix R is constructed by reading off how a 90° clockwise rotation permutes the four quadrant positions in the bit-string ordering $(00, 01, 11, 10)$. The resulting matrix is:

$$R = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (3)$$

The four bases are given by:

$$1|\text{ref}\rangle \quad R^1|\text{ref}\rangle \quad R^2|\text{ref}\rangle \quad R^3|\text{ref}\rangle \quad (4)$$

But, because $|\text{ref}\rangle$ does not change throughout the calculations, this shorthand notation $R^i|\text{ref}\rangle \equiv |R^i\rangle$ with $i \in \{0, 1, 2, 3\}$ will be used. The usefulness of this notation is that the action of R becomes:

$$R|R^i\rangle = |R^{i+1 \bmod 4}\rangle \quad (5)$$

with $R^4 = 1$.

2×2 motifs can be represented as a superposition of the four bases:

$$|\phi\rangle = \sum_{i=0}^3 c_i |R^i\rangle, \quad (6)$$

where the coefficients vector c_i encodes the shape of the motif and its components have to follow the clockwise labelling of pixels within a 2×2 cell.

6.2. Constructing MPS from Geometric Primitives

Since the quantics encoding applies the same 2×2 subdivision recursively, the rotation basis defined above can be used as the local basis at every site in the MPS chain. This means a full image can be described by assigning one of these local states to each site. For an image that is made up of repeated motifs, there are only three building blocks. Each site in the chain is assigned

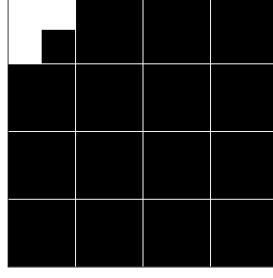
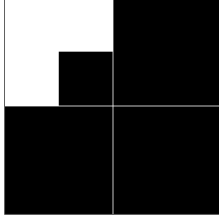
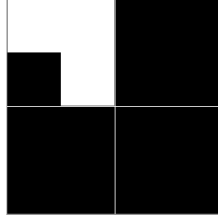


Figure 11: **An L-shaped motif in a 4×4 grid.**

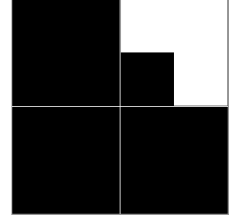
Building on the rotational basis of Fig. 10, any sites of the MPS are represented by a local state $|\phi\rangle = \sum_{i=0}^3 c_i |R^i\rangle$. The coefficient vector $c_i = (c_0, c_1, c_2, c_3)$ encodes which pixels are active in that site. An L-shaped motif state $|M\rangle$ is defined with $c_M = (1, 1, 0, 1)$, while a position-selecting state $|P\rangle$ uses $c_P = (1, 0, 0, 0)$. This selects only the top-left cell at that level of the hierarchy. Together they encode the shape and location of the motif in the grid. To encode the motif illustrated in the figure, two copies of $|P\rangle$ are placed in front of $|M\rangle$ in the MPS chain. The result is a product state $|\psi\rangle = |P\rangle \otimes |P\rangle \otimes |M\rangle$.



(a) $|\psi\rangle = |P\rangle \otimes |M\rangle$



(b) $|\psi\rangle = |P\rangle \otimes R|M\rangle$



(c) $|\psi\rangle = R|P\rangle \otimes R|M\rangle$

Figure 12: **Applying the rotation operator R .**

All three panels show a 4×4 grid containing the same L-shaped motif. (a) Original position of the motif. (b) Rotation operator R is applied at site 2 (the motif site): the primitive shape is rotated by 90° clockwise within its 2×2 cell. The position in the 4×4 grid is unchanged. (c) R is applied to site 1. Because site 1 encodes which quadrant of the grid is occupied, rotating at that site shifts the entire motif to the adjacent quadrant. This acts like a translation by half the grid width. Neither operation increases the bond dimension. The representation of the motif remains a product state.

one of three states: motif $|M\rangle$, position $|P\rangle$ and filler $|F\rangle$. These are states given by equation 6 with a specific choice of coefficients vector. Different combinations of these states across the chain produce different motif arrangements in the image grid.

The motif state $|M\rangle$ defines the geometric feature of the motif. The position state $|P\rangle$, which is placed at a coarser level than $|M\rangle$, encodes where in the grid the motif appears. The filler state $|F\rangle$ is used for scaling motifs, and its application is explained in the next section.

As an example (Fig. 11), let the motif be "L" shape, with coefficient vector $c_M = (1, 1, 0, 1)$. The motif state is going to be written as (using equation 6):

$$|M\rangle = |R^0\rangle + |R^1\rangle + |R^3\rangle \quad (7)$$

To place this in the top left corner of a 8×8 grid, set two position states with a coefficient vector $c_P = (1, 0, 0, 0)$ in front of the MPS. The resulting state encoding such image is:

$$|\psi\rangle = |P\rangle \otimes |P\rangle \otimes |M\rangle \quad (8)$$

Note this is an MPS with bond dimension one, hence a product state.

6.3. Motif Manipulation: Rotation, Scaling and Quadrant Size Translation

Applying R to either the position state or the motif state produces different motif arrangements in the image (Fig. 12).

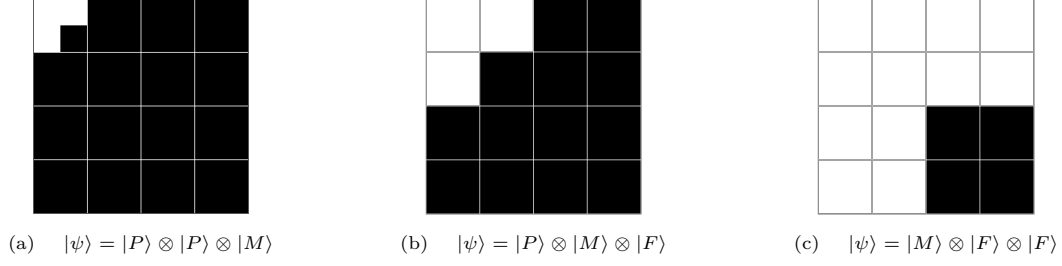


Figure 13: **Moving the motif tensor to a coarser site makes the motif larger.**

All three panels show the same L-shaped primitive on a 8×8 grid. Size is not set by an operator. It is set by which site in the MPS the $|M\rangle$ state is placed at. A filler state $|F\rangle$ is placed at every finer site after it. $|F\rangle$ fills all pixels within each selected cell uniformly. Moving $|M\rangle$ from the finest site to the coarsest produces the same shape at sizes 2×2 , 4×4 , and 8×8 . The bond dimension does not increase.

Consider a 4×4 grid, encoded as a 2 site MPS. Place the same "L" motif state described in the previous section in the top left corner of the grid. This state is described by:

$$|\psi\rangle = |P\rangle \otimes |M\rangle \quad (9)$$

Applying R to the second site rotates pixels within the 2×2 block, leaving the position of the motif in the top left quadrant (Fig. 12 (b)). The new state with the rotated motif is written as:

$$|\psi\rangle = |P\rangle \otimes R|M\rangle \quad (10)$$

Applying R to the position state translates the motif by a quadrant size (Fig. 12 (c)). The orientation of the motif stays the same. This can be thought of as a Ferris wheel-like rotation.

Scaling is not implemented by an operator. It is instead a choice of which level in the MPS chain the motif tensor $|M\rangle$ is placed at (Fig. 13). This process requires a third building block, the filler state $|F\rangle$. This is a uniform superposition over all four basis states, so it "activates" all sub-cells given by the coarser site. Scaling is carried out by placing $|M\rangle$ at a coarser site and $|F\rangle$ at every finer site after it. This way the coarse site determines the overall shape and $|F\rangle$ fills in all pixels within each selected cell uniformly.

To see this concretely, consider the three rescalings illustrated in Fig. 13. Moving $|M\rangle$ from finer site to coarser sites, with $|F\rangle$ filling all sites below it, produces the same L-shaped primitive with size: 2×2 , 4×4 , and 8×8 . These are encoded as:

$$\begin{aligned} |\psi_1\rangle &= |P\rangle \otimes |P\rangle \otimes |M\rangle \\ |\psi_2\rangle &= |P\rangle \otimes |M\rangle \otimes |F\rangle \\ |\psi_3\rangle &= |M\rangle \otimes |F\rangle \otimes |F\rangle \end{aligned}$$

6.4. Single-Pixel Translation

6.4.1. Factorisation and Why Single-Pixel Translation Breaks It

A state is factorisable when its features at each scale are independent of one another and can be written as a tensor product:

$$|\Psi\rangle = \left(\sum_{i=0}^3 \alpha_i |R^i\rangle^{[1]} \right) \otimes \left(\sum_{j=0}^3 \beta_j |R^j\rangle^{[2]} \right) \otimes \cdots \otimes \left(\sum_{k=0}^3 \gamma_k |R^k\rangle^{[N]} \right) \quad (11)$$

This means that a pattern at a coarse site gives no constraint on the pattern at a finer site. When entanglement is present between sites, the bond dimension is greater than one and scales are coupled. All the states used in the previous section were factorisable states.

Single-pixel translation breaks this factorisation. When a motif is shifted by one pixel across the boundary between two adjacent 2×2 cells, the encoding at the coarser site changes. Consider

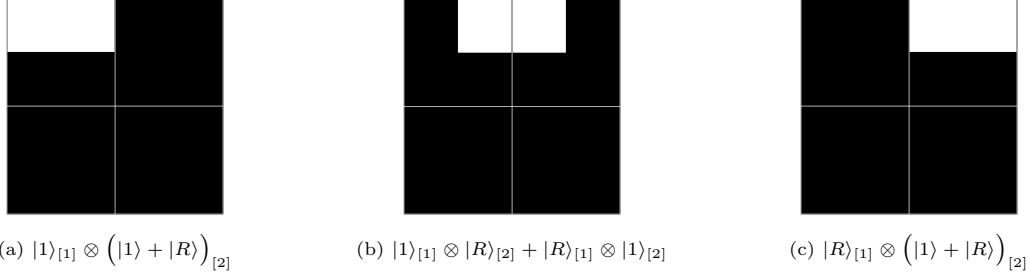


Figure 14: **Shifting a motif by one pixel increases the bond dimension.**

All three panels show a two pixel horizontal bar on a 4×4 grid. The subscript denotes the site in the MPS. **(a)** Both pixels sit within the same 2×2 cell. The state has bond dimension 1, it is a product state. **(b)** The bar is shifted by one pixel to the right. The motif crosses a boundary, coupling sites 1 and 2. The bond dimension grows to 2 as the features at site 2 are no longer factorisable. **(c)** After a second shift, the motif now sits fully within the next 2×2 cell. The state is a product again, since the bond dimension returns to 1.

a motif on a 4×4 grid, shaped as a 2-pixel horizontal bar (Fig. 14). Initially encoded as a product state:

$$|1\rangle_{[1]} \otimes (|1\rangle + |R\rangle)_{[2]} \quad (12)$$

where the subscripts denote the MPS sites. Moving the motif by one pixel to the right produces the following encoding:

$$|1\rangle_{[1]} \otimes |R\rangle_{[2]} + |R\rangle_{[1]} \otimes |1\rangle_{[2]} \quad (13)$$

Now the right pixel of the motif has crossed a site 1 boundary, and this causes the site 1 to change from $|1\rangle_{[1]}$ to $|R\rangle_{[1]}$. Site 2 now takes a different value depending on site 1, so the sites are no longer independent and the state cannot be written as a product. The MPS has to be written with a bond dimension of two. Moving the motif again by one pixel gives the state:

$$|R\rangle_{[1]} \otimes (|1\rangle + |R\rangle)_{[2]} \quad (14)$$

which is a product state. Hence, moving motifs across boundaries will fluctuate the bond dimension of their encoding; therefore, an explicit MPO construction is required.

6.4.2. Constructing the Single-Pixel Translation MPO

The key observation is that x -translation reduces to a one-dimensional problem over the column bits. Shifting right increases only the column bits while leaving the row bits unchanged. The column bits form a binary counter across the MPS chain from fine to coarse, so shifting by one pixel is equivalent to incrementing this counter by one. This propagates carry bits from the finest site to the coarsest. The same logic is implemented in a ripple-carry adder [27]. Bou-Comas et al. [28] showed that this structure can be encoded as an MPO with bond dimension 2, working in a binary quantum encoding. This construction is extended to the base-4 encoding. The full derivation is given in Appendix D.

The MPO is constructed from two local operators, defined as:

$$T_{\text{nc}}^x = |R\rangle\langle 1| + |R^2\rangle\langle R^3| \quad (15)$$

$$T_c^x = |1\rangle\langle R| + |R^3\rangle\langle R^2| \quad (16)$$

T_{nc} represents the “no carry” operation, this is applied when pixels are in the first column of the 2×2 cell. These pixels, when moved to the right, do not affect the coarser site, since they stay within the cell. T_c represents the “carry” operation. This is applied to pixels that are in the second column of the 2×2 cell. When these pixels are moved to the right, they overflow to the next cell, so the coarser site encoding has to be changed.

The MPO for a motif translated in the x direction on a 8×8 grid is given by:

$$\mathcal{T}_x = (\mathbb{1} \otimes \mathbb{1} \otimes T_{\text{nc}}) + (\mathbb{1} \otimes T_{\text{nc}} \otimes T_c) + (T_{\text{nc}} \otimes T_c \otimes T_c) \quad (17)$$

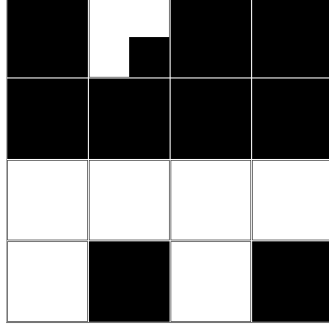


Figure 15: **Multiple instances of the same motif can be stored more compactly.**

The image shows an 8×8 grid with three instances of the same L-shaped motif at different positions and sizes. Each copy is encoded as a separate MPS with bond dimension 1. The two scaled copies at the bottom share the same tensors from site 2 onwards. These identical tails do not need to be stored twice. They are merged into a single branch. The total bond dimension of the combined state drops from 3 to 2. No information is lost.

The three terms correspond to three carry scenarios. The first term operates on pixels that are in the first column bits in the finest scale, site 3 tensor. No carry has to be emitted from this site, since pixels do not cross any boundaries. The second term operates on pixels that are in the second column of the finest site. These pixels will cross boundaries created by site 2, and hence the encoding has to change in the coarser site. The third term operates on pixels that are in the second column of both the finest and the middle site. These pixels cross two cell boundaries simultaneously: first from the finest 2×2 cell to the next, and then the boundary made by the 4×4 quadrants. A carry must therefore propagate through both site 3 and site 2. This is why T_c appears at both the middle and the coarsest site, while T_{nc} at the coarsest site handles the final column increment without further overflow. It is important to note that MPO bond dimension is 2 regardless of grid size. This holds because each site in the MPO must only track whether the column bit at the previous site overflowed, which is a single binary choice.

The MPO for y translation has the same topology as $\mathcal{T}_x$, the only difference is that the gate has to be replaced by the row-increment operator, defined as:

$$T_{nc}^y = |R^3\rangle\langle 1| + |R^2\rangle\langle R| \quad (18)$$

$$T_c^y = |R\rangle\langle R^2| + |1\rangle\langle R^3| \quad (19)$$

6.5. Suffix Compressibility of Images Constructed from Motifs

The translation MPO derived above applies to a single motif instance. In practice, an image may contain the same motif at multiple positions or scales and can be represented as a superposition of MPS. The following section shows that such superpositions are compressible.

The compression is done by identifying which MPS in the sum share identical suffixes and merging those tails into a single branch. Two MPS are suffix-equivalent at site i if all their tensors from site i to the end of the chain are identical. Only the tensors before the shared site differ.

This compressibility is best explained by a direct example (Fig. 15). Consider a repeated motif placed in an 8×8 grid. Set different scales for it, and place 3 $|M\rangle$ in the grid. Arrangement in Fig. 15 can be written as three MPS:

$$|\psi_1\rangle = |P\rangle \otimes R|P\rangle \otimes |M\rangle \quad (20)$$

$$|\psi_2\rangle = R^2|P\rangle \otimes |M\rangle \otimes |F\rangle \quad (21)$$

$$|\psi_3\rangle = R^3|P\rangle \otimes |M\rangle \otimes |F\rangle \quad (22)$$

Using the block scheme described in Appendix B.1, the sum state is written as:

$$A^{[1]} = (|P\rangle \quad R^2|P\rangle \quad R^3|P\rangle) \quad A^{[2]} = \begin{pmatrix} R|P\rangle & 0 & 0 \\ 0 & |M\rangle & 0 \\ 0 & 0 & |M\rangle \end{pmatrix} \quad A^{[3]} = \begin{pmatrix} |M\rangle \\ |F\rangle \\ |F\rangle \end{pmatrix} \quad (23)$$

States 2 and 3 share the same MPS suffix at sites 2 and 3. So these can be factorised and the superposition of the two is accumulated on site 1 as such:

$$|\psi_1\rangle = |P\rangle \otimes R|P\rangle \otimes |M\rangle \quad (24)$$

$$|\psi_2\rangle = (R^2 + R^3)|P\rangle \otimes |M\rangle \otimes |F\rangle \quad (25)$$

This compresses the sum state from bond dimension 3 to 2. Importantly, the compression is lossless.

7. Discussion

Class masks: The class mask extracted from the Quantics MPO classifier closely matches the statistical pixel-mean baseline computed directly from the training data. This means that the classifier captures something close to a weighted class average. However, this baseline does not illustrate how features are organised across spatial scales. The Quantics MPO can.

Scale-separated features: At fine scales, the 4×4 motifs are nearly identical across all 10 classes. This implies that primitives in each class are universal and do not possess any class-discriminating information. On the other hand, 8×8 features are class-dependent and possess class-discriminating information. This scale-dependent structure has a direct analogy in the renormalisation group (RG) framework [29, 30]. This is an iterative coarse-graining procedure that allows feature extraction from a physical system by examining different length scales. Mehta and Schwab [31] showed that RG and deep learning separate features in a coarse-grained way. They distinguish relevant operators, which are the parts of the operator that perform the discrimination of the classifier, from irrelevant operators, which have no effect on the classifier decisions. The coarse-scale bonds (sites 1-2-3) carry the class-discriminating information and are therefore the relevant part of the MPO. The fine-scale bonds (sites 4-5) contribute nothing to the classification and are irrelevant in the RG sense.

Interpretability of extracted features: Individual motifs extracted at the fine scale are difficult to interpret as recognisable geometric patterns. This may be a direct consequence of the encoding. As noted by Wright et al. [1], the loss of locality in amplitude encoding means that the physical representation of the entanglement structure is not straightforward to interpret. The encoded correlations appear to be more global in nature, where the most significant singular values capture global structure rather than spatially localised features. Therefore, the extracted features from the classifier are only partially interpretable.

Fine-scale redundancy and compressibility: Truncating fine-scale bonds in each class MPS sometimes improves accuracy. This indicates that the removed components contain noise rather than useful signal. Replacing the last two site tensors of every class MPS with those of class 0 drops overall accuracy by only 0.5%. This confirms that a shared fine-scale motif dictionary is feasible.

Geometric operations are cheap: The fact that rotation, scaling, and quadrant translation do not increase bond dimension has a direct consequence for classifier compression. It means that a single motif can be reused at many positions, orientations, and scales without increasing the complexity of the MPS representation. This supports the idea that a compact motif library combined with placement rules could represent complex images efficiently.

Single-pixel translation is costly: Single-pixel translation is more expensive, but the cost is bounded. The translation MPO has bond dimension 2 regardless of grid size, which means that the entanglement does not grow with image resolution. This is promising for scaling to larger images.

Connecting the two phases: The two phases of this work are complementary and are directed towards a more efficient construction of the Quantics MPO classifier. Phase one establishes that fine-scale motifs are universal. Phase two establishes that such a motif can be manipulated across the image grid relatively easily. Together, these results suggest that the classifier's complexity is concentrated in the coarse-scale bonds, and that the fine-scale structure is geometrically manageable.

7.1. Limitations

The results of Section 5.3 confirm that the fine-scale features are universal, but they do not indicate whether the specific motif from class 0 is the optimal representative motif. A more rigorous test would be to check whether replacing motifs from other classes produces the same accuracy drop. If all classes give the same result, universality is confirmed more consistently.

The geometric manipulation investigation only handles 90° rotations. Rotations by arbitrary angles would spread a single pixel’s intensity across multiple output pixels unevenly. Encoding this in the quantics basis would be significantly more complex and likely require higher bond dimensions. These limitations do not contradict the main conclusions but they constrain how far the current framework can be implemented in practice.

Why the classifier plateaus at 81% accuracy: The classifier assigns a label by contracting a test image with a single MPO. This returns a vector of ten scores, one per class. The class with the highest overlap wins.

The construction has two steps that make up the prototypes. First, SVD compression discards the weakest correlations within each class. This means that the prototype captures the dominant structure of the class, not the full average. Second, a polar decomposition is applied to the MPO. This forces the ten class prototypes into an orthonormal set. The result is an overlap classifier with ten fixed orthogonalised prototypes. Its accuracy is limited by how well each prototype represents its class. This is a problem for classes with high within-class variation. For example, a "7" with a horizontal slash and one without both belong to the same class, but only one is close to the prototype. This limitation cannot be solved by increasing the bond dimension, since one prototype per class cannot represent a variable distribution. The compression results from Section 5.3 reinforce this interpretation. The most compact bond profile (4, 3, 2, 1) loses almost no accuracy. For some classes aggressive truncation even improves the accuracy by about 10%, indicating that the components discarded at higher bond dimensions encode noise rather than discriminative structure.

By contrast, the variational MPS classifier of Stoudenmire and Schwab [25] achieves 97% accuracy on the same dataset because its optimisation can learn decision boundaries that separate overlapping classes, rather than relying on a single fixed prototype per class.

8. Conclusion

This work investigated whether a deterministic tensor network classifier built on Quantics encoding can offer transparency into its decision-making process and whether understanding its internal structure can lead to a more efficient construction.

Two results were achieved. First, coarse-scale MPO bonds encode class-discriminating structure while fine-scale bonds are universal across all MNIST classes. Replacing the two finest sites across all classes drops accuracy by only 0.5%, indicating that a universal, small-scale primitive exists in the classifier. Second, an algebraic framework was derived showing that rotation, scaling, and quadrant translation of Quantics encoded motifs are cheap operations that preserve the product-state structure. However, single-pixel translation requires a bond dimension 2 MPO, bounded independently of grid size. Together, these results suggest that the classifier’s complexity is concentrated in a small number of coarse-scale tensors and that the fine-scale structure is both universal and geometrically manageable.

8.1. Improvements and Future Work

Combining the two phases: The most significant open direction is merging the two phases. This can be done by first extracting the dominant shared motif from the classifier. Then reconstruct each class prototype by placing this motif at different positions, scales, and orientations across the image grid. The suffix-merging result from Section 6.5 guarantees that MPS sharing identical fine-scale suffixes collapse into a single branch in the sum state. If a small number of placements per class is sufficient, the entire classifier reduces to a shared motif plus a set of rules encoding

where and how that motif is utilised per class. This would give a fully interpretable and geometric based alternative to the current MPO.

Exploiting fine-scale redundancy through downscaling: In this work the original 28×28 MNIST images are upsampled to 32×32 images. This produces a 5-site MPS. Section 5.3 showed that the last two sites are highly redundant and a single shared motif at this scale is sufficient. This raises the question of whether upscaling to 32×32 is necessary at all. Constructing the classifier on downsampled 16×16 images would yield a 4-site MPS, removing the redundant fine sites entirely. In addition to gaining computational efficiency, working at a coarser resolution may expose different patterns. This would test whether the universality result observed at the finer scale is a property of the data or a derivative of the chosen resolution.

8.2. Acknowledgement

I would like to thank Prof. Andrew G. Green for his supervision throughout this project, for his guidance on the direction of the work, and for his advice regarding the structure and presentation of this report.

I would also like to thank PhD candidate Ivan Shalashilin for his help in understanding the codebase underlying the Quantics MPO classifier, and for pointing to the forum discussion that suggested the similarity between the class masks and the statistical baseline.

The code used to extract features, and produce all figures in this work is available at [32].

References

- [1] L. Wright, F. Barratt, J. Dborin, V. Wimalaweera, B. Coyle, A. G. Green, [Deterministic Tensor Network Classifiers](#), arXiv:2205.09768 (May 2022). doi:10.48550/arXiv.2205.09768.
- [2] A. Krizhevsky, I. Sutskever, G. E. Hinton, [ImageNet Classification with Deep Convolutional Neural Networks](#), in: F. Pereira, C. J. Burges, L. Bottou, K. Q. Weinberger (Eds.), Advances in Neural Information Processing Systems, Vol. 25, Curran Associates, Inc., 2012.
- [3] Y. Lecun, L. Bottou, Y. Bengio, P. Haffner, [Gradient-based learning applied to document recognition](#), Proceedings of the IEEE 86 (11) (1998) 2278–2324. doi:10.1109/5.726791.
- [4] C. Rudin, [Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead](#), Nature Machine Intelligence 1 (5) (2019) 206–215. doi:10.1038/s42256-019-0048-x.
- [5] R. Orus, [A Practical Introduction to Tensor Networks: Matrix Product States and Projected Entangled Pair States](#), arXiv:1306.2164 (Jun. 2014). doi:10.48550/arXiv.1306.2164.
- [6] J. C. Bridgeman, C. T. Chubb, [Hand-waving and Interpretive Dance: An Introductory Course on Tensor Networks](#), Journal of Physics A: Mathematical and Theoretical 50 (22) (2017) 223001, arXiv:1603.03039 [quant-ph]. doi:10.1088/1751-8121/aa6dc3.
- [7] J. Biamonte, V. Bergholm, [Tensor Networks in a Nutshell](#), arXiv:1708.00006 (Jul. 2017). doi:10.48550/arXiv.1708.00006.
- [8] H. W. Lin, M. Tegmark, D. Rolnick, [Why does deep and cheap learning work so well?](#), Journal of Statistical Physics 168 (6) (2017) 1223–1247, arXiv:1608.08225 [cond-mat]. doi:10.1007/s10955-017-1836-5.

- [9] I. Goodfellow, Y. Bengio, A. Courville, Deep learning, Adaptive computation and machine learning, The MIT press, Cambridge, Mass, 2016.
- [10] A. Tomut, S. S. Jahromi, A. Sarkar, U. Kurt, S. Singh, F. Ishtiaq, C. Muñoz, P. S. Bajaj, A. Elborady, G. d. Bimbo, M. Alizadeh, D. Montero, P. Martin-Ramiro, M. Ibrahim, O. T. Alaoui, J. Malcolm, S. Mugel, R. Orus, [CompactifAI: Extreme Compression of Large Language Models using Quantum-Inspired Tensor Networks](#), arXiv:2401.14109 [cs] (May 2024). [doi:10.48550/arXiv.2401.14109](#).
- [11] H.-M. Rieser, F. Köster, A. P. Raulf, [Tensor networks for quantum machine learning](#), Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences 479 (2275) (2023) 20230218. [doi:10.1098/rspa.2023.0218](#).
- [12] S.-J. Ran, [Encoding of matrix product states into quantum circuits of one- and two-qubit gates](#), Physical Review A 101 (3) (2020) 032310. [doi:10.1103/PhysRevA.101.032310](#).
- [13] R. Penrose, Applications of Negative Dimensional Tensors in Combinatorial Mathematics and its Applications (1971).
- [14] G. Evenbly, [A Practical Guide to the Numerical Implementation of Tensor Networks I: Contractions, Decompositions, and Gauge Freedom](#), Frontiers in Applied Mathematics and Statistics 8 (2022) 806549. [doi:10.3389/fams.2022.806549](#).
- [15] G. H. Golub, C. F. Van Loan, Matrix computations, 4th Edition, Johns Hopkins studies in the mathematical sciences, Johns Hopkins Univ. Press, Baltimore, MD, 2013.
- [16] D. Perez-Garcia, F. Verstraete, M. M. Wolf, J. I. Cirac, [Matrix Product State Representations](#), arXiv:quant-ph/0608197 (May 2007). [doi:10.48550/arXiv.quant-ph/0608197](#).
- [17] U. Schollwöck, [The density-matrix renormalization group](#), Reviews of Modern Physics 77 (1) (2005) 259–315. [doi:10.1103/RevModPhys.77.259](#).
- [18] G. Vidal, [Efficient classical simulation of slightly entangled quantum computations](#), Physical Review Letters 91 (14) (2003) 147902, arXiv:quant-ph/0301063. [doi:10.1103/PhysRevLett.91.147902](#).
- [19] M. Hauru, M. V. Damme, J. Haegeman, [Riemannian optimization of isometric tensor networks](#), SciPost Physics 10 (2) (2021) 040, arXiv:2007.03638 [quant-ph]. [doi:10.21468/SciPostPhys.10.2.040](#).
- [20] M. Rath, H. Date, [Quantum Data Encoding: A Comparative Analysis of Classical-to-Quantum Mapping Techniques and Their Impact on Machine Learning Accuracy](#), arXiv:2311.10375 [quant-ph] (Nov. 2023). [doi:10.48550/arXiv.2311.10375](#).
- [21] N. Munikote, [Comparing Quantum Encoding Techniques](#), arXiv:2410.09121 [quant-ph] (Oct. 2024). [doi:10.48550/arXiv.2410.09121](#).

- [22] J. Green, J. B. Wang, [Quantum Encoding of Structured Data with Matrix Product States](#), arXiv:2502.16464 [quant-ph] (Dec. 2025). doi:10.48550/arXiv.2502.16464.
- [23] J. I. Latorre, [Image compression and entanglement](#), arXiv:quant-ph/0510031 (Oct. 2005). doi:10.48550/arXiv.quant-ph/0510031.
- [24] I. V. Oseledets, [Approximation of \$2^d \times 2^d\$ Matrices Using Tensor Decomposition](#), SIAM Journal on Matrix Analysis and Applications 31 (4) (2010) 2130–2145. doi:10.1137/090757861.
- [25] E. M. Stoudenmire, D. J. Schwab, [Supervised Learning with Quantum-Inspired Tensor Networks](#), arXiv:1605.05775 (May 2017). doi:10.48550/arXiv.1605.05775.
- [26] danbst, [How does a simple logistic regression model achieve a 92% classification accuracy on MNIST?](#), published: Cross Validated.
- [27] Sarah L. Harris and David Harris, [Digital Design and Computer Architecture](#), Elsevier, 2022. doi:10.1016/C2019-0-00213-0.
- [28] A. Bou-Comas, M. Płodzień, L. Tagliacozzo, J. J. García-Ripoll, [Quantics Tensor Train for solving Gross-Pitaevskii equation](#), arXiv:2507.03134 [cond-mat] (Jul. 2025). doi:10.48550/arXiv.2507.03134.
- [29] K. G. Wilson, [The renormalization group and the \$\epsilon\$ expansion](#), Physics Reports 12 (2) (1974) 75–199. doi:10.1016/0370-1573(74)90023-4.
- [30] K. G. Wilson, [The renormalization group and critical phenomena](#), Reviews of Modern Physics 55 (3) (1983) 583–600. doi:10.1103/RevModPhys.55.583.
- [31] P. Mehta, D. J. Schwab, [An exact mapping between the Variational Renormalization Group and Deep Learning](#), arXiv:1410.3831 [stat] (Oct. 2014). doi:10.48550/arXiv.1410.3831.
- [32] A. Rodano, [aleksej-rodano/4th_year_project_submitted_code](#), original-date: 2026-04-04T14:47:53Z (Apr. 2026).

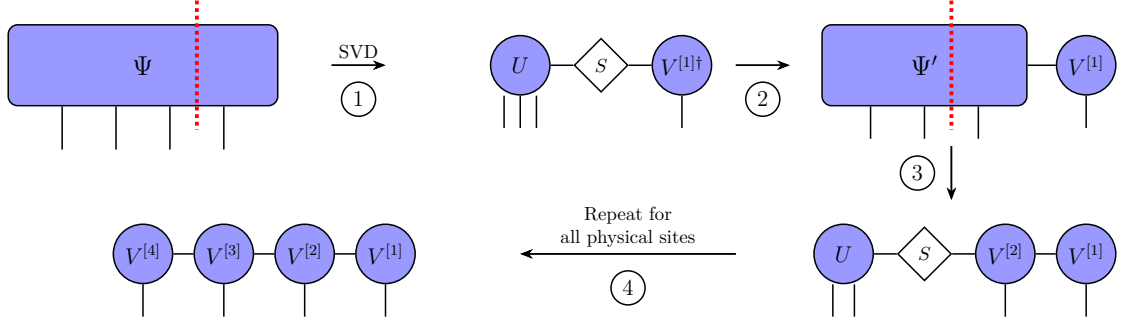


Figure A.16: **From dense state to matrix product state.**

(1) The state is reshaped into a matrix and then SVD is applied. (2) The left singular vectors and singular values are multiplied together to form a remainder state Ψ' , keeping the right singular vectors $V^\dagger$ as the first MPS tensor. (3) The remainder is then reshaped into a matrix, and SVD is applied again. (4) Repeating this for each physical index of the original state produces an MPS in right-canonical form. In such a form, the norm of the state is concentrated on the leftmost site, and contracting any other site with its complex conjugate gives the identity.

Appendix A. From SVD to Matrix Product State

Appendix A.1. Singular Value Decomposition

Singular value decomposition (SVD) [15] is the central tool used throughout this work. This allows to decompose exactly any matrix M as

$$M = USV^\dagger \quad (\text{A.1})$$

where U is a unitary matrix with orthonormal columns, V is a unitary matrix with orthonormal rows, and S is a diagonal matrix containing r non-zero singular values in descending order. The number of non-zero singular values equals the rank of M .

The singular values measure how much information each component of the decomposition carries. This makes the SVD the perfect candidate for compressing [17]. Truncating to the $r' < r$ largest singular values gives an approximation:

$$M \approx U_{r'} S_{r'} V_{r'}^\dagger, \quad (\text{A.2})$$

where $U_{r'}$ and $V_{r'}^\dagger$ are the first r' columns and rows of U and $V^\dagger$ respectively, and $S_{r'}$ is the $r' \times r'$ upper block of S .

Appendix A.2. Constructing MPS

The SVD introduced above can be used to decompose a high-rank tensor into a chain of smaller tensors, known as a matrix product state (MPS). The full procedure is illustrated in Fig. A.16. This MPS is constructed following the procedure of Vidal [18]. The result is a chain of rank-3 tensors $A^{[k]}$, one per site k . Each tensor possesses one physical index σ_k and two bond indices connecting it to its neighbours.

The procedure described in the figure produces an MPS in right-canonical form. This means that every site tensor except the first satisfies the right-isometry condition:

$$\sum_{\sigma_i} V_{\sigma_i} (V_{\sigma_i})^\dagger = \mathbb{1} \quad (\text{A.3})$$

Appendix B. MPS Summation and Maximum Bond Dimension

Appendix B.1. Block-Diagonal Summation of MPS

This appendix illustrates how multiple MPS are combined into a single sum state using a block-diagonal scheme, adapted from Wright et al. [1]. Consider creating a sum state with three MPS: $\mathcal{A}$, $\mathcal{B}$ and $\mathcal{C}$.

- First site: concatenate as a row vector

$$\mathcal{A}_{\sigma_1}^{[1]} = \left(\underline{A}_{\sigma_1}^{[1]}, \underline{B}_{\sigma_1}^{[1]}, \underline{C}_{\sigma_1}^{[1]} \right) \quad (\text{B.1})$$

- Internal site: tensors placed on block diagonal

$$\mathcal{A}_{\sigma_n}^{[n]} = \begin{pmatrix} \underline{A}_{\sigma_n}^{[n]} & 0 & 0 \\ 0 & \underline{B}_{\sigma_n}^{[n]} & 0 \\ 0 & 0 & \underline{C}_{\sigma_n}^{[n]} \end{pmatrix} \quad (\text{B.2})$$

- Final site: concatenated as a column vector

$$\mathcal{A}_{\sigma_N}^{[N]} = \begin{pmatrix} \underline{A}_{\sigma_N}^{[N]} \\ \underline{B}_{\sigma_N}^{[N]} \\ \underline{C}_{\sigma_N}^{[N]} \end{pmatrix} \quad (\text{B.3})$$

where $\underline{A}$ and $\underline{B}$ denote a vector and a matrix, respectively.

Appendix B.2. Maximum Theoretical bond dimension in the Quantics MPO Classifier

The Schmidt rank sets a hard ceiling on the bond dimension of any MPS. At each bond k , the bond dimension is bounded by the dimension of the smaller of the two subsystems on either side of the cut:

$$D_k \leq \min(\dim \mathcal{H}_{\text{left}}, \dim \mathcal{H}_{\text{right}}) \quad (\text{B.4})$$

This ceiling holds for any state, including superpositions, and gives the minimum bond dimension needed to represent the state exactly.

The Quantics MPO classifier built on 32×32 MNIST images has 5 sites with physical dimension 4, and a label index of dimension 10 on the first site. At each bond cut (from left to right), the right subsystem contains only physical indices, giving dimensions 256, 64, 16, 4. The left subsystem contains both physical and label indices, giving dimensions 40, 160, 640, 2560. Taking the minimum at each cut gives the theoretical maximum bond profile of this Quantics MPO, which is the following:

$$(D_1, D_2, D_3, D_4) = (40, 64, 16, 4) \quad (\text{B.5})$$

Appendix C. 8×8 Masks Extracted from the Quantics MPO Classifier

The 8×8 masks are extracted using the same procedure as the 4×4 motifs in Section 5.2. The difference is that the orthogonality centre is placed at site 3 rather than site 4. Contracting the left bond of site 3 with a one-hot vector and then contracting all the virtual indices on sites 3, 4 and 5 produces an 8×8 mask.

The left bond of site 3 has dimension 16, meaning that 16 masks exist at this scale. The four most dominant masks for each class are shown in Fig. C.17. Unlike the 4×4 case, the singular values do not decay as rapidly, hence more patterns are required to capture the class information.

The overlap for the four most dominant masks for each class is shown in Fig. C.18. The 8×8 masks show very little overlap across classes, meaning each class has a distinct set of coarse features.

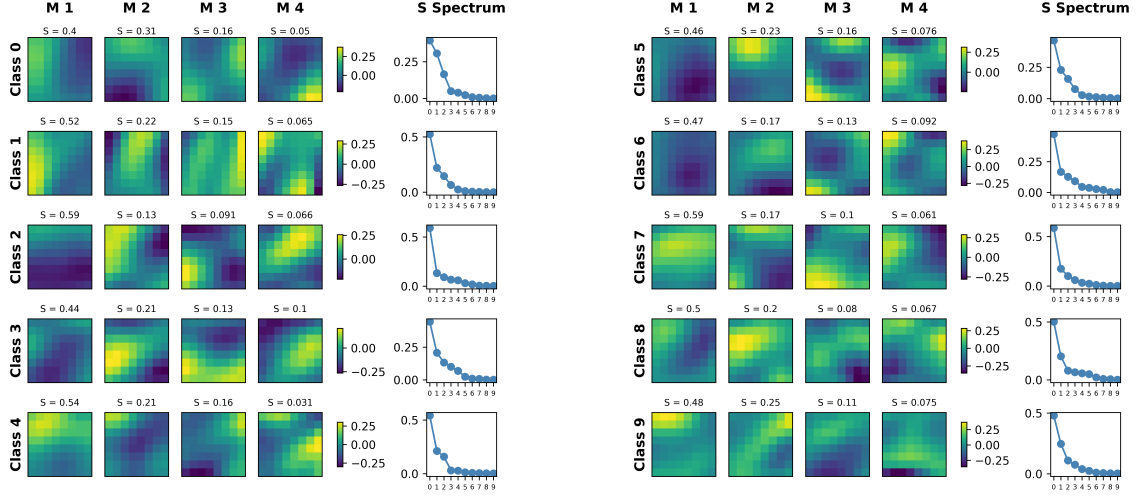


Figure C.17: **The four most dominant 8×8 masks per digit class.** These are extracted by placing the orthogonality centre at site 3 of each per-class prototype MPS. Contracting the left bond of site 3 with a one-hot vector and then contracting the site 3, 4, 5 tensors produces an 8×8 mask. The singular value spectrum of site 3 is illustrated on the right of each row.

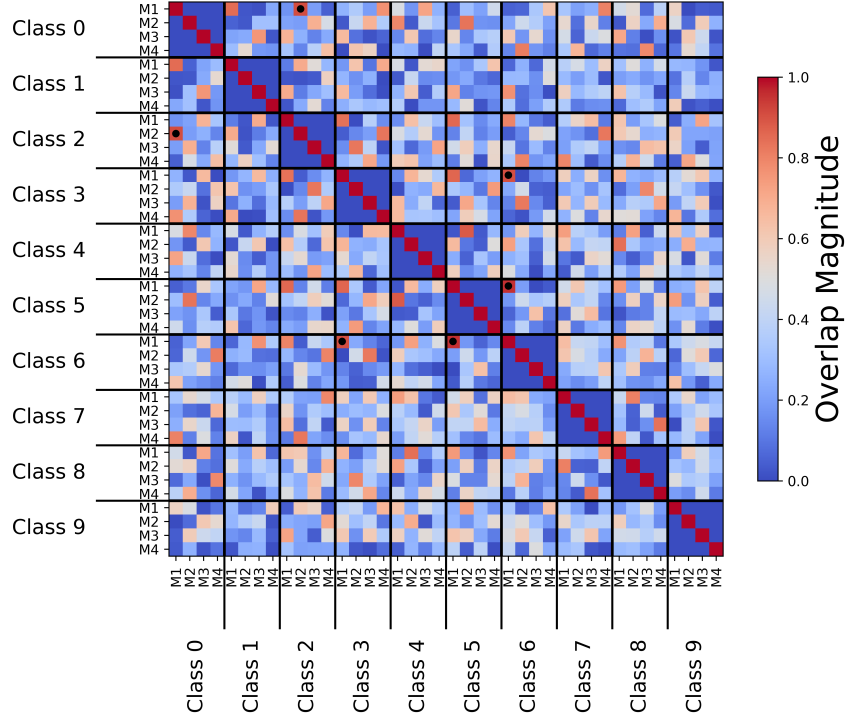


Figure C.18: **Overlap heatmap for 8×8 masks.**

Overlap of masks illustrated in Fig. C.17. Unlike the 4×4 motifs, there is very little overlap across classes, with some exceptions. This means that each class has its own distinct coarse features. Motifs overlap higher than 0.99 are denoted by a black dot (excluding the diagonal, since it is trivially $\text{diag}(1, \dots, 1)$).

Appendix D. MPO Construction for Single-Pixel Translation via Carry Arithmetic

This appendix derives the single pixel x -translation MPO for a quantics MPS on an 8×8 grid. The construction follows the ripple-carry adder framework of Bou-Comas et al. [28]. This is adapted to the base-4 clockwise encoding used in this work.

General Translation MPO: Any translation by an amount a can be written as an MPO acting on an N -site quantics MPS. The general form is (taken from [28]):

$$\mathcal{T}_a = \sum_{s_i, s'_i} T_{s_1, s'_1, l_1}^{a_1} T_{l_1, s_2, s'_2, l_2}^{a_2} \cdots T_{l_{N-1}, s_N, s'_N}^{a_N} |\{s'_i\}\rangle\langle\{s_i\}| \quad (\text{D.1})$$

where $a = \sum_k a_k 2^{-k}$ is the shift amount written in binary. The labels s_k, s'_k are the input and output physical indices at site k , and $l_k \in \{0, 1\}$ are the virtual (carry) indices connecting adjacent sites. The carry propagates from the finest site to the coarsest. Each tensor T^{a_k} encodes the arithmetic at one site.

The Local Arithmetic Rule: At each site k , the tensor T^{a_k} implements a binary addition step. It takes as inputs three values: the incoming carry l_{in} , the shift digit a_k , and the column bit of the input state $c(s_k)$. It returns two values: an output column bit and an outgoing carry. The column bit $c(s)$ of each basis state is defined as $c(|1\rangle) = c(|R^3\rangle) = 0$ and $c(|R\rangle) = c(|R^2\rangle) = 1$. The row bit is unaffected by the x direction shift. The arithmetic rules are:

$$\text{sum} = a_k + c(s) + l_{\text{in}} \quad (\text{D.2})$$

$$c(s') = \text{sum} \bmod 2 \quad (\text{D.3})$$

$$l_{\text{out}} = \left\lfloor \frac{\text{sum}}{2} \right\rfloor \quad (\text{D.4})$$

Computing the Local Gate Operators: Each site tensor T^{a_k} can be treated as a 2×2 matrix in the carry indices:

$$W_{l_{\text{out}}, l_{\text{in}}}^{[k]} = \begin{pmatrix} W_{0,0}^{[k]} & W_{0,1}^{[k]} \\ W_{1,0}^{[k]} & W_{1,1}^{[k]} \end{pmatrix} \quad (\text{D.5})$$

where each entry is a physical operator:

$$W_{l_{\text{out}}, l_{\text{in}}}^{[k]} = \sum_{s, s'} T_{l_{\text{out}}, s, s', l_{\text{in}}}^{a_k} |s'\rangle\langle s| \quad (\text{D.6})$$

The non-zero entries of T^{a_k} are those consistent with the arithmetic rules defined in equations D.3 and D.4. For an 8×8 grid, the grid is encoded as a 3-site MPS. For a one-pixel x -shift, $a_3 = 1$ at the finest site and $a_1 = a_2 = 0$ at the coarser sites. The boundary condition at the finest site is $l_{\text{in}} = 0$; since there is no finer site to produce an incoming carry. The arithmetic calculations for the finest site are summarised in Table D.1 and calculations for the coarser site where the column increment is zero (i.e. $a_1 = a_2 = 0$) are summarised in Table D.2.

Two types of operation are determined. For basis states $|1\rangle$ and $|R^3\rangle$, no carry is emitted during their translation ($l_{\text{out}} = 0$). These transform as:

$$|1\rangle \rightarrow |R\rangle \quad |R^3\rangle \rightarrow |R^2\rangle \quad (\text{D.7})$$

which can be written as operators:

$$T_{\text{nc}} = |R\rangle\langle 1| + |R^2\rangle\langle R^3| \quad (\text{D.8})$$

Let us call this a no-carry operator as it does not propagate a carry unit.

For translation of basis states $|R\rangle$ and $|R^2\rangle$, a carry is emitted ($l_{\text{out}} = 1$). These transform as:

$$|R\rangle \rightarrow |1\rangle \quad |R^2\rangle \rightarrow |R^3\rangle \quad (\text{D.9})$$

Table D.1: **Arithmetic rules for the finest site (site 3) for one-pixel x-shift.**

Columns l_{in} and l_{out} are the incoming and outgoing carry bits, $c(s)$ and $c(s')$ are the column bits of the input and output basis states, and $a_3 = 1$ is the shift digit. The incoming carry is fixed to $l_{\text{in}} = 0$, since no finer site exists. Each row shows how an input basis state is mapped to an output basis state. The basis states $|1\rangle$, $|R^3\rangle$, which occupy the first column of the clockwise 2×2 basis defined in Fig. 10, do not produce a carry ($l_{\text{out}} = 0$). This is described by the no-carry operator $T_{\text{nc}} = |R\rangle\langle 1| + |R^2\rangle\langle R^3|$. Basis states $|R\rangle$ and $|R^2\rangle$, which occupy the second column, overflow to the next cell, producing a carry ($l_{\text{out}} = 1$). This is described by the carry operator $T_c = |1\rangle\langle R| + |R^3\rangle\langle R^2|$.

l_{in}	a_3	$c(s)$	sum	$c(s')$	l_{out}	Physical transition	Gate
0	1	0	1	1	0	$ 1\rangle \rightarrow R\rangle, R^3\rangle \rightarrow R^2\rangle$	T_{nc}
0	1	1	2	0	1	$ R\rangle \rightarrow 1\rangle, R^2\rangle \rightarrow R^3\rangle$	T_c

Table D.2: **Arithmetic table for the coarser sites (sites 1 and 2) for a one-pixel x-shift.**

The shift digit is $a_k = 0$ at these sites, so no shift is applied directly. Both values of l_{in} are accessible here. When $l_{\text{in}} = 0$, the basis state is unchanged. When $l_{\text{in}} = 1$, an incoming carry shifts the basis state, following the same gate logic as table D.1.

l_{in}	$a_{1,2}$	$c(s)$	sum	$c(s')$	l_{out}	Physical transition	Gate
0	0	0	0	0	0	$s' = s$	$\mathbb{1}$
0	0	1	1	1	0	$s' = s$	$\mathbb{1}$
1	0	0	1	1	0	$ 1\rangle \rightarrow R\rangle, R^3\rangle \rightarrow R^2\rangle$	T_{nc}
1	0	1	2	0	1	$ R\rangle \rightarrow 1\rangle, R^2\rangle \rightarrow R^3\rangle$	T_c

This gives the *carry* operator:

$$T_c = |1\rangle\langle R| + |R^3\rangle\langle R^2| \quad (\text{D.10})$$

Assembling the W Matrices: Substituting T_{nc} and T_c gates (with the help of table D.1) into equation D.6, gives the W matrix at each site. At the finest site, the boundary condition $l_{\text{in}} = 0$ eliminates the right column:

$$W^{[3]} = \begin{pmatrix} T_{\text{nc}} & 0 \\ T_c & 0 \end{pmatrix} \quad (\text{D.11})$$

At the coarser sites, from Table D.2:

$$W^{[2]} = \begin{pmatrix} \mathbb{1} & T_{\text{nc}} \\ 0 & T_c \end{pmatrix} \quad (\text{D.12})$$

The first site in the chain has a boundary condition. No carry can be emitted from it as there is no coarser site, hence the second row in the matrix vanishes:

$$W^{[1]} = \begin{pmatrix} \mathbb{1} & T_{\text{nc}} \\ 0 & 0 \end{pmatrix} \quad (\text{D.13})$$

Contracting the W matrices along the carry indices gives:

$$\mathcal{T}_x = \sum_{l_1, l_2} W_{l_1}^{[1]} \otimes W_{l_1, l_2}^{[2]} \otimes W_{l_2}^{[3]} \quad (\text{D.14})$$

Expanding over $l_1, l_2 \in \{0, 1\}$ gives four terms. The term $l_1 = 1, l_2 = 0$ is zero because $W_{1,0}^{[2]} = 0$. The surviving terms are:

$$l_1 = 0, l_2 = 0 : \quad \mathbb{1} \otimes \mathbb{1} \otimes T_{\text{nc}} \quad (\text{D.15})$$

$$l_1 = 0, l_2 = 1 : \quad \mathbb{1} \otimes T_{\text{nc}} \otimes T_c \quad (\text{D.16})$$

$$l_1 = 1, l_2 = 1 : \quad T_{\text{nc}} \otimes T_c \otimes T_c \quad (\text{D.17})$$

Summing these yields the result written in Section 6.4.2:

$$\mathcal{T}_x = (\mathbb{1} \otimes \mathbb{1} \otimes T_{\text{nc}}) + (\mathbb{1} \otimes T_{\text{nc}} \otimes T_c) + (T_{\text{nc}} \otimes T_c \otimes T_c) \quad (\text{D.18})$$